# Deal Intelligence Agent

Antigravity build workflow for HackwithHyderabad 3.0 (Hindsight memory hackathon)

This document is a step-by-step process for building the Deal Intelligence Agent inside Google Antigravity. It includes the rules to paste, the prompts for each build phase, the verification steps and a time plan. Antigravity's menus change between versions, so treat this as a process and adapt the exact clicks to your version.

## 1. What to know about Antigravity first

- Antigravity is an agent-first IDE (a VS Code fork) that Google shipped in November 2025. Its Manager view lets you dispatch goals in plain language and review the results.
- It communicates through **Artifacts**: Task Lists, Implementation Plans and Walkthroughs capture the agent's intent and proof, so you verify in minutes instead of reading raw logs.
- **Rules** are system-level instructions applied automatically. **Workflows** are saved prompts you trigger with `/` commands. Both are under the Customizations menu.
- Google announced Antigravity 2.0 in May 2026 and split the product into several surfaces. One guide says the Agent Manager may eventually leave the IDE for a separate app.
- Sources disagree on the rules folder name (.agent/ vs .antigravity/). Create rules through the Customizations menu instead of depending on a path.

## 2. Phase 0: Preparation (30 minutes, before any prompt)

1. Create an empty folder `deal-intel-agent` and run `git init`. Commit after every phase so you can roll back.
2. Prepare accounts and keys: Hindsight (Cloud with promo code MEMHACK99 added in the billing section after registering, or local Docker), a Groq API key, and Band AI desktop installed (required by the hackathon).
3. Put keys in a `.env` file and create `.gitignore` containing `.env` **before** the agent writes any code.
4. Save the hackathon problem statement PDF as `docs/problem_statement.pdf`.
5. Give the agent the Hindsight docs. The Hindsight project offers a docs skill for coding agents (`npx skills add https://github.com/vectorize-io/hindsight --skill hindsight-docs`). If Antigravity does not load it, save the docs pages into `docs/hindsight/` as markdown.
6. Set Antigravity's permissions to ask before running terminal commands until the project structure exists. Avoid letting an agent run destructive shell commands.

## 3. Phase 1: Rules (paste once)

Create a workspace rule with the following content. Also add a short project instruction file restating the architecture.

```
PROJECT: Deal Intelligence Agent for a hackathon. Judged on: Innovation 30%,
Hindsight memory 25%, Technical implementation 20%, UX 15%, Real-world impact 10%.
STACK: Python 3.11, Streamlit UI, Hindsight (hindsight-client) for ALL long-term
memory, Groq API for the LLM (models: openai/gpt-oss-120b, qwen/qwen3-32b).
RULES:
1. Never invent Hindsight API parameters. Read docs/hindsight/ first. If unsure, say so.
2. Every Hindsight call goes through agent/memory.py only.
3. Every LLM call goes through agent/llm.py, which must retry, validate JSON,
   and fall back gracefully. Models can fail at function calling.
4. Never hardcode secrets. Read from .env.
5. LLM outputs that state facts about a deal must cite the memory they came from.
   If memory is empty, say "no history yet", never guess.
6. Keep scope tight: pre-call brief, post-call debrief, outcome learning loop, alerts.
```

```
7. Write type hints, docstrings, and one test per module.
8. After each task, produce a Walkthrough artifact listing what changed and how it
   was verified.
```

## 4. Phase 2: Spec and plan (Planning mode)

Open the Manager, create a workspace named "Deal Intel", add an agent and choose **Planning mode**. The agent will then produce an Implementation Plan for you to review before it writes code.

### Prompt 1: spec interview (the highest-value step)

```
Read docs/problem_statement.pdf. I am building the "Deal Intelligence Agent"
idea. Before writing any code, interview me with one question at a time to
produce SPEC.md. Cover: user persona, the 3 core flows, the demo story
(60 seconds), what "memory visibly helping" looks like on screen, and
what is explicitly out of scope. Challenge vague answers.
```

### Prompt 2: architecture plan

```
From SPEC.md, create an Implementation Plan with: folder structure, module
responsibilities, data schemas (extracted deal JSON, memory sentence format),
memory bank layout (one bank per deal + one shared company-playbook bank),
and a build order. Flag any assumption about the Hindsight API you can't
confirm from docs/hindsight/.
```

**Review the plan:** check that memory appears in every flow, the scope is small, and the agent flagged unknown API details instead of guessing.

## 5. Phase 3: Build in dependency order

Run steps 3.1 to 3.3 one after another because everything depends on them. Step 3.4 (data) can run in a parallel agent. Steps 3.5 onward can be split across agents once the foundation works.

### 3.1 Smoke test

```
Create scripts/smoke_test.py that: (1) retains one sentence into a test bank
in Hindsight, (2) recalls it, (3) calls Groq once and prints the response.
Run it. Report exact errors if any. Do not build anything else.
```

Do not continue until this works. It catches key and setup problems early.

### 3.2 Reliable LLM wrapper

```
Build agent/llm.py: a Groq client with (a) retry with exponential backoff,
(b) strict JSON parsing with schema validation, (c) on invalid output,
re-prompt the model with the error, up to 3 times, (d) a clear fallback.
Add tests that simulate malformed JSON and API timeouts.
```

### 3.3 Memory wrapper

```
Build agent/memory.py wrapping Hindsight: retain_deal_event(deal_id, text, date),
recall_deal(deal_id, question), retain_playbook(text), recall_playbook(question),
reflect_playbook(question). One bank per deal, one 'company-playbook' bank.
Use only API calls documented in docs/hindsight/.
```

### 3.4 Synthetic data (parallel agent)

```
Create scripts/generate_data.py using the LLM to produce 10 realistic B2B
SaaS deals: 6 closed (4 won, 2 lost), 4 open. Each deal needs 4-8 dated
interactions (call transcripts, emails). Use realistic names, companies, prices,
and 3-4 recurring competitors. Embed patterns on purpose, for example:
pricing objections from CFOs close more often when ROI case studies are used;
deals stall at legal review. Save as JSON in data/synthetic/. Also create a
ground_truth.json listing the embedded patterns so we can test whether the
agent discovers them.
```

The ground_truth.json file lets you show that the agent discovered patterns you planted, which is strong evidence for technical implementation and memory value.

### 3.5 Extraction pipeline

```
Build agent/extract.py: transcript -> JSON (objections, competitors,
stakeholders with sentiment, pricing, commitments with due dates, next steps).
Convert each item into a dated, plain-English memory sentence naming the
person and company. Before retaining, recall the existing related fact and
detect contradictions (e.g. budget changed); retain it as an explicit update.
Skip duplicates using a content hash.
```

### 3.6 Pre-call brief

```
Build agent/brief.py: given a deal_id, run several recall queries (open
objections, stakeholders, unfulfilled commitments, competitors, last contact),
plus playbook recall for similar situations. Return a structured brief with
a "memories used" list. Also implement a no-memory mode that sends the same
request to the LLM without any recalled context, for before/after comparison.
```

### 3.7 Learning loop

```
Build agent/playbook.py: on deal closed (won/lost), summarize tactics used,
objections faced, and outcome; retain to company-playbook; then call reflect
to form cross-deal patterns. Add a function that returns current learned
insights. Test it against data/synthetic/ground_truth.json.
```

### 3.8 Alerts

```
Build agent/alerts.py: scan open deals via memory for overdue commitments,
stakeholders silent for 14+ days, and deals stalled at the same stage
longer than similar past deals. Return prioritized alerts with evidence.
```

### 3.9 UI (after 3.5 to 3.8 exist)

```
Build app.py in Streamlit with 5 pages: deal list with risk flags,
Brief Me (with a Memory ON/OFF toggle and a "memories used" side panel),
Log a Call (shows extracted facts and what changed), Alerts, and
Playbook Insights. Prioritize clarity over decoration.
```

## 6. Phase 4: Verification

Run these as separate prompts so the agent tests its own work.

```
Write and run tests: (1) smoke test of the full flow on one deal,
(2) contradiction handling, (3) empty-memory behavior (must say "no history"),
(4) malformed LLM output, (5) duplicate ingestion. Report failures honestly.
```

```
Use the browser to open the Streamlit app and walk through the 60-second demo
script in SPEC.md. Capture screenshots and note anything confusing or broken.
```

Antigravity has an integrated browser and the agent publishes screenshots and recordings as Artifacts, which gives you demo-flow QA for free.

## 7. Phase 5: Submission deliverables

```
Write README.md: problem, architecture diagram, an explicit "How Hindsight
memory is used" section (retain/recall/reflect, bank layout, before/after
example), setup steps, and demo instructions.
```

Have a separate agent draft the article, social post and video script from the README, then edit them yourself so they sound like you. Check the official content guide for exact requirements.

Required submission items from the problem statement: GitHub repository with clean documented code, demo video, live demo to judges, content deliverables (article, social post, video) from every team member, and an explanation of how Hindsight memory is used.

## 8. Tips for the best results

- **Review artifacts and plans, not logs.** That is where Antigravity saves you time.
- **One task per agent.** Small, well-defined prompts beat "build the whole app".
- **Model choice.** Gemini 3 Pro is the default. Claude and GPT-OSS options are available if an agent keeps losing the thread on a complex multi-file change. This is only for coding. Your app still calls Groq for its own LLM work.
- **Say what "done" means** in each prompt (for example "run it and report the output").
- **Commit after each phase.**
- **Do not let the agent guess the Hindsight API.** Rule 1 exists for this reason.
- **Seed demo data into Hindsight early** with a seed_memory.py script so the demo does not depend on live ingestion.

## 9. Time plan

| Block | Work |
|---|---|
| Hour 0-2 | Phase 0, rules, spec interview, plan |
| Hour 2-5 | Smoke test, LLM wrapper, memory wrapper, synthetic data (parallel) |
| Hour 5-12 | Extraction, brief, learning loop, alerts |
| Hour 12-18 | UI, then verification and bug fixing |
| Final hours | README, demo recording, content deliverables |

## 10. Target project structure

```
deal-intel-agent/
  app.py                  # Streamlit UI
  agent/
    memory.py             # wrappers around Hindsight retain/recall/reflect
    llm.py                # Groq calls + retry/validation
    extract.py            # transcript -> structured JSON -> memory sentences
    brief.py              # pre-call brief (memory ON/OFF modes)
    alerts.py             # overdue promises, silent stakeholders, stalls
    playbook.py           # outcome feedback + reflect
  data/synthetic/         # generated deals + ground_truth.json
  scripts/
    smoke_test.py
    generate_data.py
    seed_memory.py
  docs/                   # problem statement, Hindsight docs
  SPEC.md
  README.md
```